小~老沙老

Matrix-Chain Multiplication

DP Example: Matrix-Chain Multiplication

- If A is a p **x** q matrix and B a q **x** r matrix, then $C = AB$ is a p **x** r matrix

$$C[i, j] = \sum_{k=1}^{q} A[i, k]\, B[k, j]$$

q 乘 q 次

共有 pxr 個 elements

time complexity: $O(pqr)$

```
Matrix-Multiply(A, B)
1.  if A.columns ≠ B.rows
2.        error "incompatible dimensions"
3.  else let C be a new A.rows * B.columns matrix
4.        for i = 1 to A.rows
5.              for j = 1  to B.columns
6.                    c_ij = 0
7.                    for k = 1 to A.columns
8.                          c_ij = c_ij + a_ik b_kj
9.        return C
```

DP Example: Matrix-Chain Multiplication

- **The matrix-chain multiplication problem**
  - Input: Given a chain $\langle A_1, A_2, \ldots, A_n \rangle$ of n matrices, matrix $\boldsymbol{A_i}$ has dimension $\boldsymbol{p_{i-1}}$ **x** $\boldsymbol{p_i}$
  - Objective: Parenthesize the product $A_1 A_2 \ldots A_n$ to minimize the number of scalar multiplications

- **Exp:** dimensions: A_1: 4 **x** 2; A_2: 2 **x** 5; A_3: 5 **x** 1

  $(A_1A_2)A_3$: total multiplications = 4 **x** 2 **x** 5 + 4 **x** 5 **x** 1 = 60

  $A_1(A_2A_3)$: total multiplications = 2 **x** 5 **x** 1 + 4 **x** 2 **x** 1 = 18

- So the order of multiplications can make a big difference!

Matrix-Chain Multiplication: Brute Force

- $A = A_1 A_2 \ldots A_n$: How to compute A using the minimum number of multiplications?
- Brute force: check all possible orders?
  - $P(n)$: number of ways to multiply n matrices.

$$P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

  - $P(n) = \Omega(4^n / n^{3/2})$, **exponential** in n.
- Any efficient solution?
  - The matrix chain **is linearly ordered and cannot be rearranged!!**
  - Smell dynamic programming?

Matrix-Chain Multiplication

- $m[i, j]$: minimum number of multiplications to compute matrix $A_{i..j} = A_i A_{i+1} \dots A_j$, $1 \le i \le j \le n$
  - $m[1, n]$: the cheapest cost to compute $A_{1..n}$

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j. \end{cases}$$

$A_{i..j} = (A_i \dots A_k)(A_{k+1} \dots A_j)$

matrix A_i has dimension $p_{i-1} \times p_i$

- Applicability of dynamic programming
  - **Optimal substructure:** an optimal solution contains within its optimal solutions to subproblems
  - **Overlapping subproblem:** a recursive algorithm revisits the same problem over and over again; only $\Theta(n^2)$ subproblems

Bottom-Up DP Matrix-Chain Order

```
Matrix-Chain-Order(p)
1.  n = p.length – 1
2.  Let m[1..n, 1..n] and s[1..n-1, 2..n] be new tables
3.  for i = 1 to n
4.      m[i, i] = 0
5.  for l = 2 to n      // l is the chain length
6.      for i = 1 to n – l +1
7.          j = i + l -1
8.          m[i, j] = ∞
9.          for k = i  to j -1
10.             q = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j
11.             if q < m[i, j]
12.                 m[i, j] = q
13.                 s[i, j] = k
14. return m and s
```

小抄

2個 matrix 1個下刀位置

$i > j$ 那塊砍掉

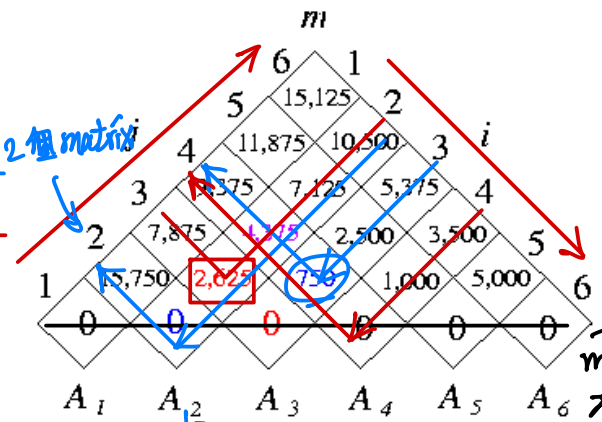
方型 → 三角型

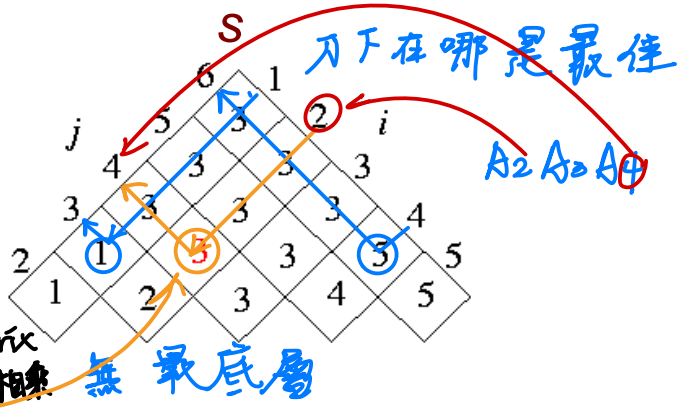
前一頁 $1 \le i \le j \le n$

A_i dimension

$p_{i-1} \times p_i$

matrix	dimension
A_1	$30 * 35$
A_2	$35 * 15$
A_3	$15 * 5$
A_4	$5 * 10$
A_5	$10 * 20$
A_6	$20 * 25$





$$m[2,4] = \min \begin{cases} m[2,2] + m[3,4] + p_1 p_2 p_4 = 0 + 750 + 35 \times 15 \times 10 = 6000. \\ m[2,3] + m[4,4] + p_1 p_3 p_4 = 2625 + 0 + 35 \times 5 \times 10 = 4375. \end{cases}$$

A2 A3 A4

切在2|3

切3|4

下在3最佳

Constructing an Optimal Solution

- $s[i, j]$: value of k such that the optimal parenthesization of $A_i A_{i+1} \ldots A_j$ splits between A_k and A_{k+1}
- Optimal matrix $A_{1..n}$ multiplication: $A_{1..s[1, n]} A_{s[1, n] + 1..n}$
- **Exp:** call Print-Optimal-Parens(s, 1, 6): $((A_1 (A_2 A_3))((A_4 A_5) A_6))$

```
Print-Optimal-Parens(s, i, j)
1. if i == j
2.         print "A"_i
3. else print "("
4.         Print-Optimal-Parens(s, i, s[i, j])
5.         Print-Optimal-Parens(s, s[i, j] + 1, j)
6.         print ")"
```

((A1 A2 A3)((A4 A5) A6)

matrix	dimension
A_1	30 * 35
A_2	35 * 15
A_3	15 * 5
A_4	5 * 10
A_5	10 * 20
A_6	20 * 25